Task 0

(.venv) anna_s@anna-465:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) anna_s@anna-465:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) anna_s@anna-465:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/anna_s/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/anna_s/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:

Task 1
(.venv) anna_s@anna-465:~/465-hw2/csce465-agentsec$ /home/anna_s/465-hw2/csce465-agentsec/.venv/bin/python /home/anna_s/465-hw2/csce465-agentsec/hw2/baseline_ctr.py
TASK 1
Command: b'{"action":"READ","path":"notes.txt"}'

-- SENDER --
Ciphertext: b'JO{\xa08\x8b\xe7\xcf\\\x05\xa9\x08\xa6\xb3\xe5\x95\xbe=-\x07\x1fv\xb4G\x19H\x16\xc2\x9c\xecn~\xbfX\xd7]'

NORMAL EXCHANGE
-- RECEIVER --
Decrypted Text: b'{"action":"READ","path":"notes.txt"}'

INTERCEPTED EXCHANGE
-- RELAY --
XOR: bytearray(b'\x10') VAL: 16
Intercepted Ciphertext:
bytearray(b'JO{\xa08\x8b\xe7\xcf\\\x05\xa9\n\xac\xa1\xf5\x95\xbe=-\x07\x1fv\xb4G\x19H\x16\xc2\x9c\xecn~\xbfX\xd7]')
-- RECEIVER --
Decrypted Text: b'{"action":"POST","path":"notes.txt"}'
(.venv) anna_s@anna-465:~/465-hw2/csce465-agentsec$

1) my design consists of a sender that encrypts text, a relay function that modifies it, and a receiver that decrypts text. the relay does not have access to the key or nonce, but the sender and receiver do. by knowing the location of the target command in the plaintext, the relay function is able to modify it to contain another command of the same length

2) in my experiment, I fed the ciphertext to the relay function, I performed xor on the READ command with the command I wanted to insert (POST) and then used that xor value to change the encrypted text to what I wanted it to baseline_ctr

3) the root cause is in the way AES-CTR works, through a sequence of encryption rounds. It preserves the order of the bits such that a character position in plaintext translates directly to a position in the ciphertext. since I knew the location of the command I wanted to change, I was able to change it by merely flipping bits without needing to decrypt anything.

Task 2
Both the gateway and node each have a long-term key pair (seen by the creation of the rsa keys in handshake.py and test.py independently of the session() call).
The Diffie-Hellman keys are established once per session because using a new secret for each one reduces the chances of an attacker learning it in time to interfere with the exchange.
Both parties sign the transcript with their private key in order to modify it according to a unique secret that would verify the authenticity of the sender and distinguish what direction communication is happening in (that way a message intended to go from node to gateway can't be bounced back). A different set of keys are used to establish authenticity and confidentiality, since if one key is exposed then the other can prevent a successful attack (an attacker who can accomplish impersonation won't be able to send a properly encrypted message, and an attacker who can crack the message encryption won't have a fake message succeed because of a separate authenticity measure).
The nonces are appended to the transcript to incorporate randomness so that the transcript hash, shared secrets, and session id don't exhibit predictable patterns. The header that is

appended to the message in the exchange contains the relevant direction and sequence information. The direction selects the operational keys for transmission; this means the same encrypted text can't be sent backwards because it would use the wrong operational keys, generated during the handshake from the new shared secret and the long-term private values. In my test, I have the expected sequence as input for the functions. I simply increase the number to demonstrate a sequence mismatch, but in a real situation, the expected sequence is automatically incremented throughout the session as valid messages go through, and the sequence is incremented with each seal() so that the same encrypted text can't be opened twice in a row; this causes the recipient to deny duplicates and protects against overwhelming transmissions.

Task 4

The attacker in these scenarios is a man-in-the-middle who is intercepting communication after messages have been encrypted and sent out, before they are received and decrypted.
Non-goals include endpoint compromise and denial of service. Endpoint compromise is pretty consistent in the test cases as they involve modifying ciphertext or headers to manipulate a good message (thus ruining it, in this case) rather than producing a fake one, and one involves the use of outright fake keys to simulate a stranger, unverified user, producing a message. DOS is present in the specific cases of tests 4 and 5, which deal with an attacker reproducing messages that have already been successfully communicated, as an approves message could simply be reproduced hundreds of times to overwhelm a recipient.
A unique IV is needed for CTR because the use of the same keystream for two different messages will result in valuable information for an attacker, as the ciphertexts can be XORed together to reveal the XOR of their corresponding plaintexts. The differences between the ciphertexts and the plaintexts will be the same since the modifications made to create the ciphertexts are the same for both. An XOR of two messages is a severe violation of the intended confidentiality; a unique IV prevents this from being possible, as the ciphertexts will have different changes from the original messages.
Verify-before-decrypt matters because it's important to ensure that the message came from the proper sender before it is processed internally. A malformed ciphertext from an attacker can cause severe issues; ensuring the communication comes from the intended person can help prevent random messages from being processed unnecessarily.
Independent encryption and mac keys are used to minimize the possibility of patterns or similarities between identity information and the message information, so as to prevent giving attackers any information that may help them. Repetition may produce patterns in data that attackers can use to determine the secret or the initial message, such as in the case of a reused IV for CTR encryption. Additionally, as mentioned previously, using the same key for both functions increases vulnerability if it's exposed, whereas using two keys means that if one is exposed, then the other's secrecy prevents attackers from acting—an imposter still can't decrypt the message, and a message forger can't convince the receiver that they are a legitimate sender.
AES-GCM uses one key for an exchange, which is less complicated and involves less overhead, but is also more vulnerable to man-in-the-middle attacks and less safe in the case of a key being leaked or exposed. It also uses the same key for encryption and authentication.

Not every entity you communicate with is guaranteed to be friendly; the sender may have a valid key that verifies they are the person who the handshake was establishe with, and the message may have arrived untouched, but a valid exchange could have happened with someone who is sending malicious content. Other security measures are needed to ensure messages are safe, and the endpoint needs local safety measures to ensure it only authorizes the proper commands.